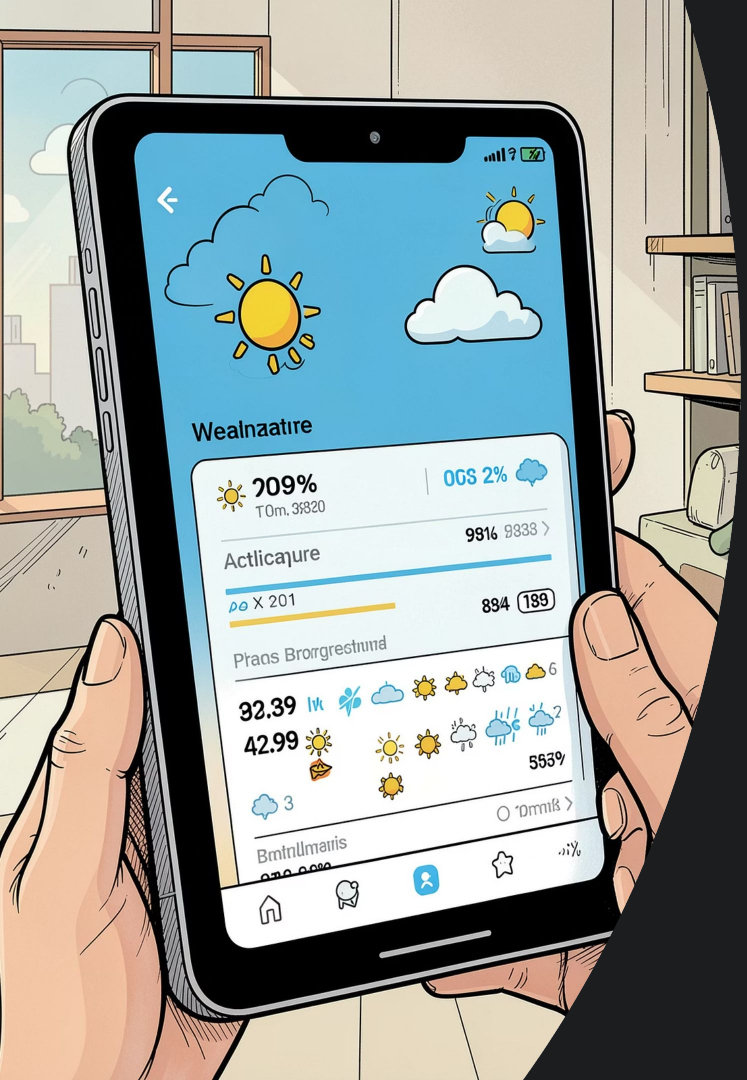
# WeatherApp

# Real-time weather updates using the Observer pattern

A Java project that demonstrates how to use the Observer design pattern to build a flexible, scalable system for monitoring weather data and distributing updates.

*Raneem Alhameed, Sidra Wali, Rahaf Atmah*



# Project Overview

WeatherApp is a Java application that fetches weather data from the **OpenWeatherMap API** and automatically distributes it to multiple observers when the data changes. The project demonstrates how to build scalable systems using design patterns.

### 🌐 API

OpenWeatherMap for fetching live weather data

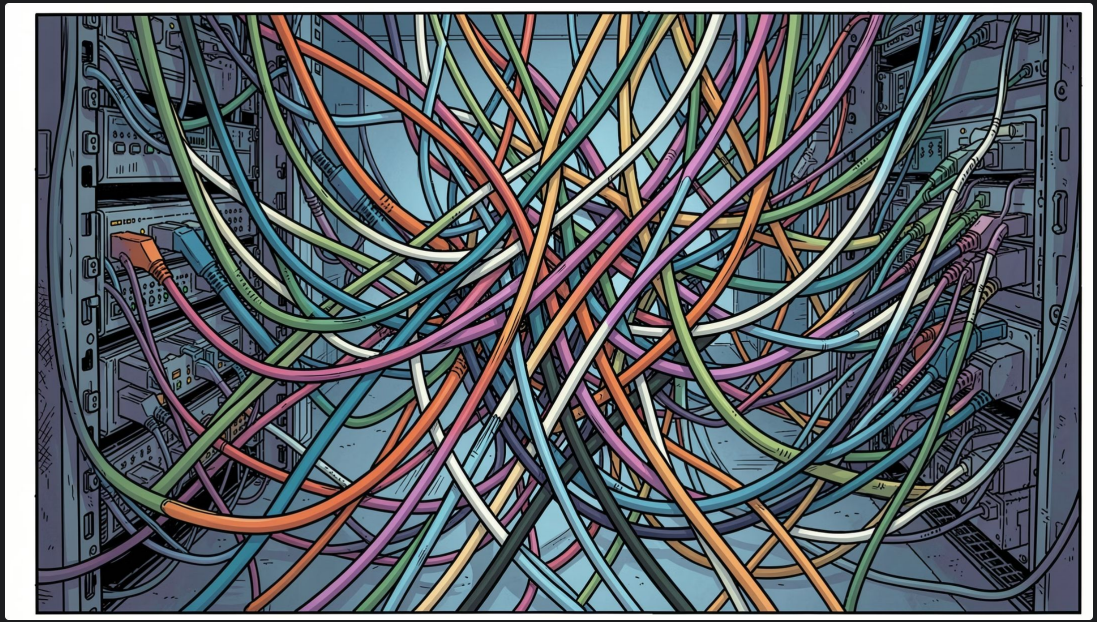
### ☕ Java

The project's primary programming language

### 🔧 Gson

Used to convert JSON into Java objects

# Problem Statement



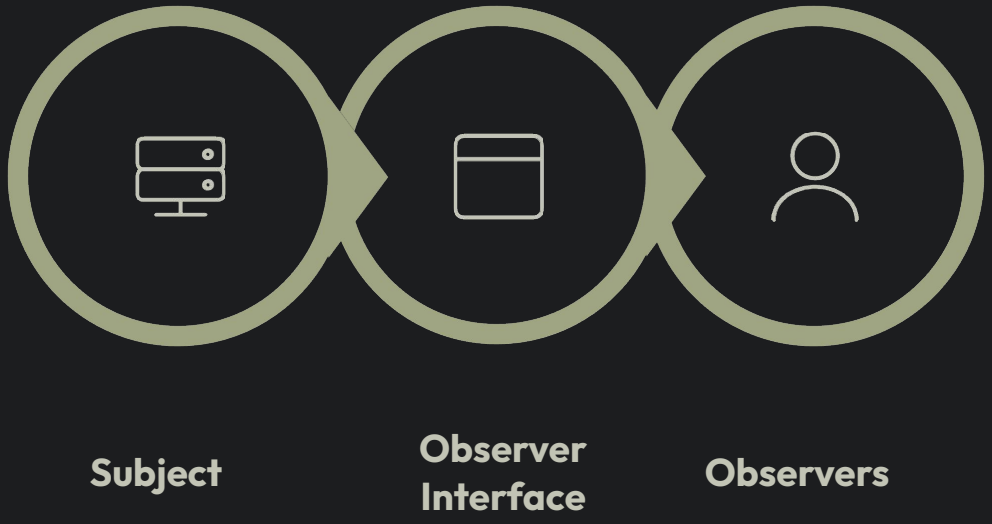
## The Main Challenge

When building a system that fetches weather data and notifies multiple parties, a key challenge emerges: **how to separate the data-fetching logic from the notification logic** without creating tight coupling between components.

- Adding a new observer requires modifying the core code
- Difficulty testing each component independently
- Lack of flexibility when notification requirements change

# Observer Pattern Principle

The Observer pattern is a behavioral design pattern that allows multiple objects (observers) to monitor another object (the subject) and respond automatically to any changes that occur — without direct coupling.




In WeatherApp: **WeatherService** notifies both **ConsoleObserver** and **EmailObserver** when weather data changes.

# Why Observer Pattern?

### Separation of Responsibilities

WeatherService is completely decoupled from the observers — each component works independently.

### High Flexibility

New observers can be added or removed without modifying the core service code.

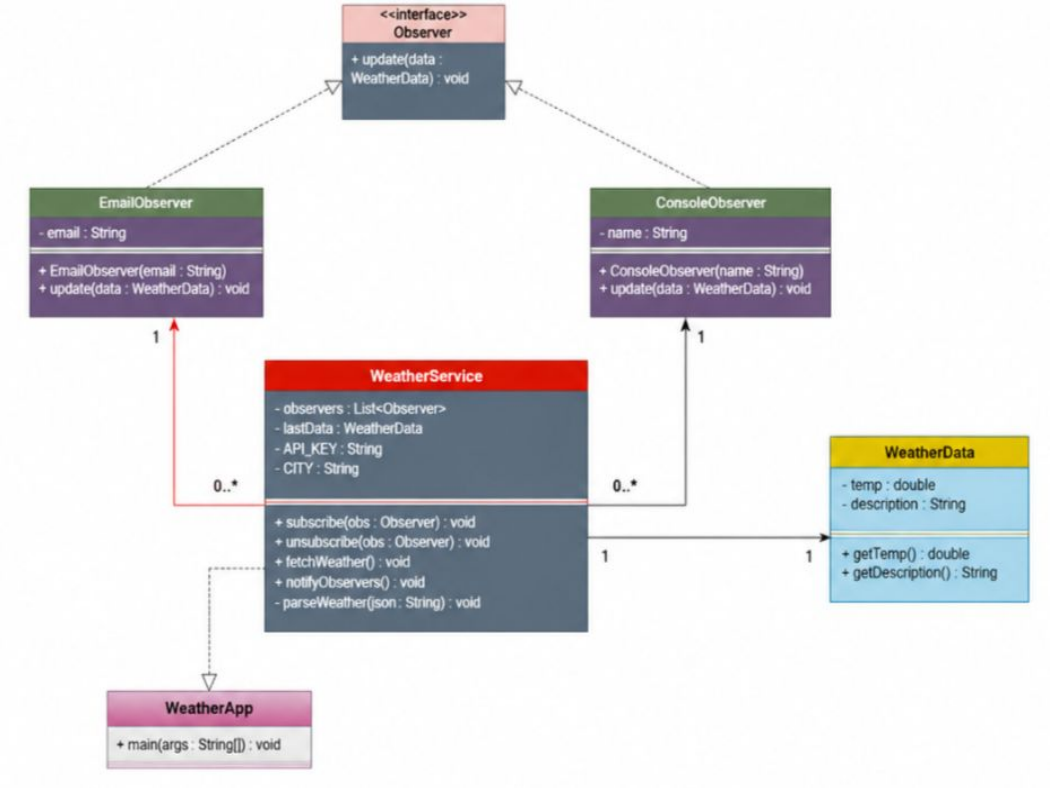
### Instant Updates

Immediate notifications are sent to all observers when data changes, efficiently.

### Easy to Test

Each observer can be tested independently without relying on the service.

# System Design and UML Diagram




### Relationships Between Classes

The diagram shows the complete system structure and the connections between its components:

- **WeatherService** — the main subject (Subject)
- **Observer** — the interface implemented by all observers
- **ConsoleObserver** — prints data in the terminal
- **EmailObserver** — sends alerts via email
- **WeatherData** — an object that holds weather data
- **WeatherApp** — the entry point and component assembler

# User Flow Scenario

Here are the steps the system goes through from the beginning until the observers are notified:

### Initialize WeatherService

Create an instance of the service with the OpenWeatherMap API key

### Subscribe Observers

Register ConsoleObserver and EmailObserver with the service

### Fetch and Parse Data

Call the API and parse the JSON response using Gson

### Notify Observers

Call update() for each registered observer when the data changes

**Next:** Let's review the implementation code together 🔍

# Summary and Key Points

### Design Flexibility

Add new observers without touching the core service code

### Easy Maintenance

Each component is independent and can be tested and modified separately

### Instant Notifications

All observers receive updates as soon as the weather data changes

### Complete Separation

Observers do not know about one another — coupling is zero between components